SiTA: Exploiting Sparsity in Tensor-Product for Accelerating Quantum Readout Error Mitigation

Hanyu Zhang[1], Zhiwei Ye[2], Liqiang Lu[1*], Kaiwen Zhou[1], Fangxu Guo[1], Siwei Tan[1]
Fangxin Liu[3], Size Zheng[4], Jianwei Yin[1]
[1]Zhejiang University [2]China Mobile (Suzhou) Software Technology Co., Ltd
[3]Shanghai Jiao Tong University [4]Tsinghua University
Email: {hyzz, liqianglu, kaiwenzhou, fangxuguo, siweitan}@zju.edu.cn yezhiwei@cmss.chinamobile.com
liufangxin@sjtu.edu.cn zhengsz@mail.tsinghua.edu.cn zjuyjw@cs.zju.edu.cn

Abstract

Though quantum computing theoretically provides exponential computational advantage, an end-to-end quantum speedup should consider the encoding, compilation, and many other peripheral process that rely on classical computer. Among them, the readout error mitigation turns out to be the bottleneck, requiring mitigation of noise errors to achieve high fidelity. To this end, leveraging the sparsity in the tensor-product has emerged as an appealing approach to improve computational efficiency. However, existing approaches, based on intermediate data pruning or threshold pruning, suffer from the limited accelerator and fidelity loss. In this paper, we propose SiTA, a quantum error mitigation accelerator that exploits the inherent sparsity of Hilbert space. The key insight is that the superposition states generated by quantum algorithms cover only a subset of the Hilbert space (*output sparsity*), and the basis state naturally exhibits bit-level sparsity (*input sparsity*). Therefore, we introduce a sparse dataflow that features probability-level and state-level parallelism, which effectively skips calculations related to zero values. Finally, we design its hardware architecture that supports various computational paradigms of tensor-product, enabling acceleration for readout error mitigation. Experiments show that SiTA achieves an end-to-end speedup of 4.9× to 1604× over prior mitigation methods, without sacrificing fidelity.

CCS Concepts

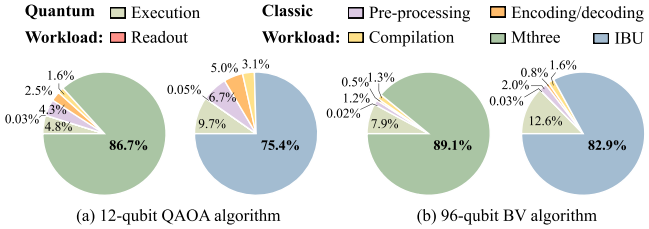
• **Hardware → Hardware accelerators**; • **Computer systems organization → Quantum computing**.

ACM Reference Format:
Hanyu Zhang[1], Zhiwei Ye[2], Liqiang Lu[1*], Kaiwen Zhou[1], Fangxu Guo[1], Siwei Tan[1], Fangxin Liu[3], Size Zheng[4], Jianwei Yin[1]. 2026. SiTA: Exploiting Sparsity in Tensor-Product for Accelerating Quantum Readout Error Mitigation. In *63rd ACM/IEEE Design Automation Conference (DAC '26), July 26–29, 2026, Long Beach, CA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3770743.3803943

*Corresponding author.

DAC '26, Long Beach, CA, USA

ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3803943



(a) 12-qubit QAOA algorithm (b) 96-qubit BV algorithm

Figure 1: Profiling of end-to-end latency using Mthree [17] and IBU [23] for quantum error mitigation.

1 Introduction

Quantum computers hold the potential to significantly accelerate certain problems, such as combinatorial optimization and machine learning. However, current quantum computers are essentially only responsible for the calculation part; the rest of the operations, such as encoding/decoding, compilation, and post-processing, still rely on classical computers [28, 30]. However, due to the substantial gap in information representation between quantum and classical computing, the workload on classical computers often significantly exceeds the runtime of quantum computers. Many so-called "quantum speedup" claims often overlook the latency on classical computers, which may fail to achieve end-to-end acceleration. *In the face of the heterogeneously decoupled quantum computer architecture, we argue that peripheral accelerators are strongly desired to fulfill the true quantum advantage.*

In the classical end, quantum readout error mitigation plays a critical role that directly determines the output fidelity. Clearly, readout errors [5, 11] are significant sources of interference, with the error rate being 51.4× greater than that of the single-qubit gate on the 127-qubit IBM Sherbrooke quantum computer. However, error mitigation operations dominate classical processing. Specifically, IBM's [17] and Google's [23] readout mitigation methods account for an average of 94.0% and 89.2% of the total classical computing time, respectively, in Figure 1. Recent studies mainly focus on improving the fidelity after error mitigation, including structure-based error mitigation methods [24, 27], deep learning-based approaches [2], and algebraic methods [17, 23, 25, 32]. Among them, the algebra-based error mitigation is a white-box and formal method, which has demonstrated effectiveness in superconducting [13], ion-trap [1], and optical [33] quantum hardware. The techniques in this direction can be further categorized into matrix-based and iterative tensor-product-based approaches. The former multiplies the noisy probability vector by a mitigation matrix to

make the distribution closer to the ideal one [17, 32], while the latter approximates this process to reduce the exponentially growing computational cost. Although the iterative tensor-product method is software-efficient and experimentally effective, it still faces a trade-off between latency and fidelity [23, 25]. For example, when using the cuBLAS [19] on NVIDIA A100 GPU, the mitigation of a 15-qubit system takes 0.4 seconds, accounting for 87.2% of the classical computing time.

On the other hand, several works have exploited sparsity to reduce computational cost and accelerate mitigation [17, 23, 25, 32]. However, these approaches are hindered by the limited speedup and low fidelity improvement. Both Mthree [17] and SpREM [32] identify the input sparsity in a structured pattern, but the fidelity improvement remains modest. In addition, IBU [23] and QuFEM [25] only consider the sparsity at the software level without the support of a domain-specific accelerator (DSA) for further speedup. For example, QuFEM improves IBU by introducing a qubit grouping scheme that generalizes the iterative tensor-product, leading to higher fidelity. However, it only eliminates the unstructured zero-valued intermediate data in the software stack, necessitating 13.4 s for mitigation. Though SpREM proposes a Hamming-sparsity format and DSA design, it lacks the characterization of the mitigation matrix, failing to achieve end-to-end speedup.

Starting with the tensor-product-based mitigation, we propose SiTA, a hardware method that exploits the inherent sparsity in quantum computing. First, we divide the general mitigation flow into three operators: matrix-vector multiplication (MVM), tensor-product (TP), and multiply-and-accumulate (MAC). Then, we recognize two sparse opportunities for acceleration, namely *output sparsity* and *input sparsity*. The output sparsity lies in the fact that, even under various noises, the readout results are only a subset of the Hilbert space. Leveraging this sparsity, we can trace back the computational flow and pre-capture the necessary operands to avoid redundant computation. The input sparsity fundamentally refers to bit-level sparsity in the basis state, e.g., $|01\rangle = [0, 0, 1, 0]$, which helps to transform the MVM operator into a column-selection operator. Based on these two sparsity patterns, we propose a sparse mitigation dataflow that features probability-level and state-level parallelism. Clearly, the TP operator essentially calculates the intermediate probability vectors, which will be gathered to generate the mitigated probability distribution. Thus, the probability-level parallelism computes these intermediate probabilities in parallel. On the other hand, the probability vector generally consists of multiple non-zero basis states. State-level parallelism means parallelizing their calculations. Finally, our accelerator architecture leverages a hierarchical memory system to efficiently store the mitigation matrices and stream the noisy probability vectors, while integrating a dynamic multiplier chain within the PE to flexibly support different parallelism configurations. The contributions of this paper are summarized as follows:

- We perform an in-depth analysis of the tensor-product-based error mitigation method to identify operator-level bottlenecks and redundant computations.
- We propose an efficient sparse error mitigation dataflow that integrates both probability-level and state-level parallelism, while fully exploiting the properties of the tensor product.

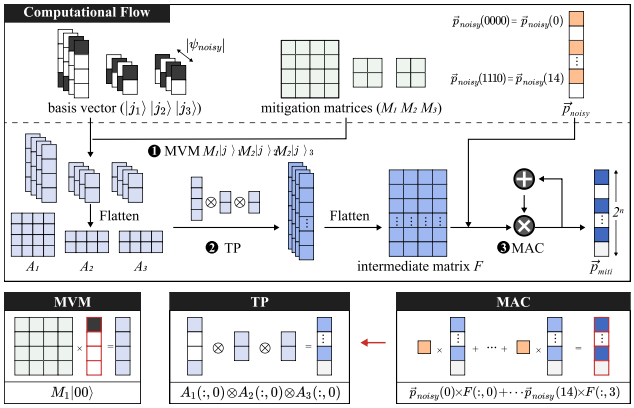


Figure 2: The computational flow of tensor-product-based error mitigation and its three essential operators.

- We design a hardware architecture that seamlessly implements our dataflow, with a dynamic multiplier chain supporting various tensor-product configurations.

Experiments show that SiTA achieves an end-to-end speedup of 4.9× to 1604× over prior mitigation methods, without sacrificing fidelity. Moreover, compared with GPU-based sparse libraries, SiTA delivers 11.2× and 5.3× speedups over the A100 and H100 GPUs, respectively.

2 Background

2.1 Matrix-Based Error Mitigation

When measuring the qubit state, the readout noise maps the ideal probability distribution $\vec{p}_{ideal}$ to a noisy one, which can be formulated as a linear transformation as follows [2–4, 16],

$$\vec{p}_{noisy} = L \cdot \vec{p}_{ideal} \tag{1}$$

The noise matrix L is a $2^n \times 2^n$ matrix, requiring 2^n measurements to characterize it, with each measurement obtaining one row of the matrix, where n is the number of qubits. Inversely, we apply such a matrix-based approach to mitigate the noise error,

$$\vec{p}_{miti} = M \cdot \vec{p}_{noisy},\ M = L^{-1} \tag{2}$$

We name the matrix M as the *mitigation matrix*, which exponentially increases with the number of qubits. The characterization and calculation of this matrix necessitate an overwhelming computational cost, exhibiting poor scalability. For example, a 30-qubit mitigation matrix would require 2,305 PB of memory, far exceeding current system capacities.

2.2 Tensor-Product-Based Error Mitigation

This method, extending from the matrix-based error mitigation, approximates the matrix as a series of tensor-product operators [23, 25]. This approach can be formulated as follows.

$$\vec{p}_{miti} = (M_1 \otimes M_2 \otimes \cdots \otimes M_k) \cdot \vec{p}_{noisy} \tag{3}$$

where $M_1, M_2, \cdots, M_k$ are the sub-mitigation-matrices derived by partitioning the qubits into k groups: $|j\rangle = |j_1\rangle\,|j_2\rangle \cdots |j_k\rangle$. For example, IBU [23] regards each qubit as a single unit, transforming the mitigation matrix as a tensor product of a series of 2×2 matrices. However, though this approach greatly reduces the arithmetic complexity, it fails to model the crosstalk between qubits, leading to

limited accuracy. This paper considers a more general formulation like QuFEM [25], which further transforms Equation (3) according to the noisy probability vector.

$$\begin{aligned}\vec{p}_{miti} &= (M_1 \otimes \cdots \otimes M_k) \sum_{j \in \psi_{noisy}} \left[\vec{p}_{noisy}(j) \, |j_1\rangle \cdots |j_k\rangle \right] \\ &= \sum_{j \in \psi_{noisy}} \vec{p}_{noisy}(j) \left[(M_1 \, |j_1\rangle) \otimes \cdots \otimes (M_k \, |j_k\rangle) \right]\end{aligned} \tag{4}$$

Here, ψ_{noisy} is the set of non-zero probability basis states in $\vec{p}_{noisy}$ after measurement. Figure 2 shows an example of a 4-qubit system with non-zero probability basis states.

$$\psi_{noisy} = \{\, |0000\rangle, |0010\rangle, |1101\rangle, |1110\rangle \,\} \tag{5}$$

By partitioning the four qubits into three groups (2,1,1), the mitigation process can be expressed as:

$$\begin{aligned}\vec{p}_{miti} = \; & \vec{p}_{noisy}(0000)(M_1 \, |00\rangle \otimes M_2 \, |0\rangle \otimes M_3 \, |0\rangle) \\ & + \vec{p}_{noisy}(0010)(M_1 \, |00\rangle \otimes M_2 \, |1\rangle \otimes M_3 \, |0\rangle) \\ & + \vec{p}_{noisy}(1101)(M_1 \, |11\rangle \otimes M_2 \, |0\rangle \otimes M_3 \, |1\rangle) \\ & + \vec{p}_{noisy}(1110)(M_1 \, |11\rangle \otimes M_2 \, |1\rangle \otimes M_3 \, |0\rangle)\end{aligned} \tag{6}$$

where $|j_1\rangle$ refers to the first two qubits, and $|j_2\rangle$, $|j_3\rangle$ refer to the $3rd$, $4th$ qubit respectively.

3 Motivation

3.1 Performance Characterization

To reduce classical computing latency and unlock the end-to-end acceleration potential of quantum computing, we analyze the computational characteristics of the iterative tensor-product method by dividing it into three tensor operators: matrix-vector multiplication (**MVM**), tensor-product (**TP**), and multiply-and-accumulate (**MAC**), as illustrated in Figure 2.

- The MVM operator refers to the multiplication between the mitigation matrix and the basis vector, e.g., $A_1(:,0) = M_1 \, |00\rangle$, $A_2(:,0) = M_2 \, |0\rangle$ in Equation (6). Considering the case of the static grouping scheme [23, 25], the overall computational complexity is determined by the non-zero probability $|\psi_{noisy}| \, O(n)$.
- The TP operator performs a tensor product among the vectors from the MVM step. Specifically, each non-zero probability state $\vec{p}_{noisy}(j)$ is responsible for a series of tensor-product, e.g., $A_1(:,0) \otimes A_2(:,0) \otimes A_3(:,0)$ in Figure 2. Each non-zero probability takes $O(n * 2^n)$ multiplications, accounting for the most computational complexity $|\psi_{noisy}| \, O(n * 2^n)$.
- The TP operator generates $|\psi_{noisy}|$ intermediate vectors that can be flatten to a matrix F. The MAC operator multiplies with the intermediate matrix to output the mitigated probability $\vec{p}_{miti}$, e.g., $\vec{p}_{noisy}(0) \times F(:,0) + \cdots + \vec{p}_{noisy}(14) \times F(:,3)$.

In Figure 3 (a), we evaluate the performance of three tensor operators using iterative tensor-product-based mitigation on CPU and GPU. Specifically, we apply the formulation in Equation (3), which employs the tensor product with a series of 4×4 matrices. Running on an AMD CPU, 0.9% are in MVM, 90.7% in TP, and 8.4% in MAC. Since we apply 4×4 matrices to model each pair of two qubits individually, MVM takes less computational cost than TP. The GPU platform achieves hundreds of speedups and shows a similar trend, where MVM, TP, and MAC operations take up 5.6%, 92.6%, and 1.8% of the run time, respectively. In a word, the tensor-product operator accounts for the most computational pressure.

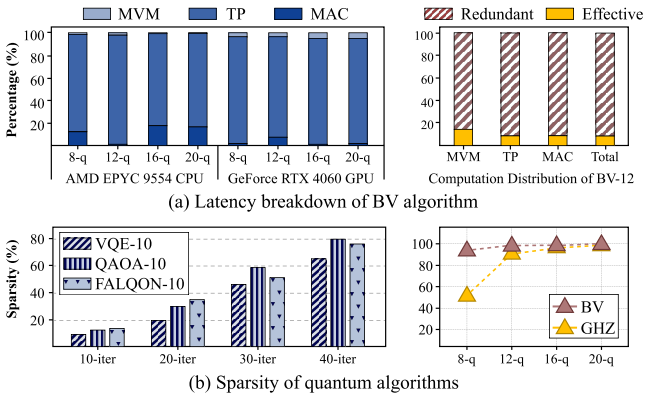


Figure 3: (a) Latency breakdown of tensor-product-based mitigation computation. (b) Sparsity of quantum algorithms.

3.2 Sparse Opportunities for Acceleration

Opportunity I: Output Sparsity. The output probability vector $\vec{p}_{miti}$ is inherently sparse as the superposition states generated by quantum algorithms cover a subset of the Hilbert space. For example, some simple quantum algorithms generate only a few non-zero probability basis states, demonstrating high sparsity — 98.6% for the 16-qubit BV algorithm and 96.7% for the 16-qubit GHZ algorithm, as shown in Figure 3 (b). Moreover, variational quantum algorithms produce increasingly sparse outputs as they converge through iterations. As shown in Figure 3 (b), after 40 iterations, the sparsity of the 10-qubit VQE, QAOA, and FALQON algorithms is 65.5%, 80.4%, and 75.3%, respectively.

Since the ideal probability distribution usually lies within the range of the noisy probability distribution, we identify the output sparsity by only calculating the elements that are non-zero probabilities in the noisy vector. Take Equation (5) as an example, since the noisy vector has four non-zero probabilities, we only calculate the corresponding elements in the mitigated vector. More importantly, pruning the mitigated vector can lead to higher accuracy as the entire probability distribution concentrates more on the scope of the ideal states. We test that by pruning the mitigated vector, the fidelity of the 12-qubit BV algorithm increases from 0.59 to 0.63. From the arithmetic level, the sparsity in the mitigated vector gives rise to complexity reduction in both TP and MAC operators, as shown in Figure 2. Specifically, by pre-obtaining the non-zeros in the mitigated vector, we can capture which elements are required to be multiplied and accumulated. In the TP operator, each non-zero element helps to guide the traceback of the necessary elements for the tensor product. As shown in Figure 3 (a), this sparsity results in 91.6% and 91.6% computational reduction for TP and MAC operators, respectively.

Opportunity II: Input Sparsity. The basis states in quantum systems are usually represented as unit vectors, where each vector only contains one '1' element with the rest of the elements equal to 0. *Thus, the input sparsity essentially is bit-level sparsity*, e.g., $|00\rangle = [1,0,0,0]$ in Figure 2. Leveraging this sparsity, the MVM

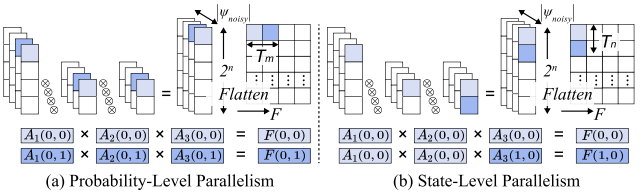


Figure 4: Two parallelizations for tensor-product operators.

operator is transformed to select one column from the matrix, determined by the non-zero index in the basis vector. As shown in Figure 3 (a), this exploitation of sparsity results in a significant 85.7% reduction in the computational load of the MVM operator. In Figure 3 (a), we also profile the total redundant computation involving both input and output sparsity, which turns out to be 91.2%. This highlights a significant potential for optimization, indicating that further improvements could significantly enhance performance and reduce unnecessary computational overhead. Therefore, this motivates the design of a specialized accelerator to exploit the sparsity in the flow of quantum readout error mitigation.

4 Sparse Dataflow for Readout Error Mitigation

4.1 Parallelization for Tensor-Product

To effectively leverage the output sparsity, we identify two orthogonal parallelisms in the TP operator, namely probability-level parallelism and state-level parallelism. As aforementioned, the TP operator calculates the intermediate probability vectors, corresponding to the number of non-zero states in the noisy probability distribution (Equation (5)). Therefore, the probability-level parallelism can be regarded as computing multiple intermediate vectors in parallel. On the other hand, each element in these vectors fundamentally represents one basis quantum state. Accordingly, state-level parallelism refers to performing parallel computation on multiple elements within an intermediate vector. By flattening the intermediate vectors, derived from the TP operator, into a $2^n \times |\psi_{noisy}|$ intermediate matrix (F), this parallelization is equivalent to tiling the output matrix and calculating the elements in parallel.

Probability-Level Parallelism. Figure 4 (a) illustrates an example of this parallelism, where two elements $F(0,0)$ and $F(0,1)$ in the intermediate matrix F are calculated in parallel. According to the rules of tensor-product, the computation of these two elements is irrelevant to each other, requiring six different elements from the input vectors. In other words, this method outputs rows of the intermediate matrix, allowing the MAC operator with a noise probability vector to generate the mitigated probability values.

State-Level Parallelism. In contrast to probability-level parallelism, this parallelization calculates the elements in the same intermediate vector, i.e., different quantum states of the same intermediate vector. Figure 4 (b) depicts an example of calculating two output elements $F(0,0)$ and $F(1,0)$ in parallel. This parallelization is responsible for generating columns of the intermediate matrix. Moreover, it is able to reuse the input elements, leveraging the inherent data reuse in the tensor-product operator. Considering these two parallelization strategies together, a tile in the intermediate matrix will be generated in one cycle. We use T_m and T_n to denote the parallelism in these two dimensions, as shown in Figure 5.

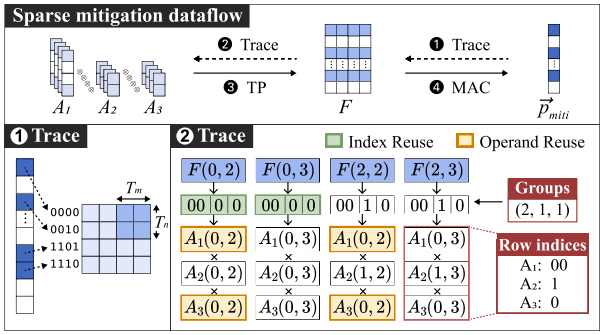


Figure 5: Sparse dataflow for quantum error mitigation.

4.2 Leveraging Output Sparsity

The parallelization mentioned before essentially is a dense dataflow. Then, we introduce its sparse version that exploits the output sparsity (opportunity I), which consists of four steps, as shown in Figure 5. ❶ We trace the non-zero values in the intermediate matrix F to gate the redundant computations derived from the sparsity of the mitigated probability vector. For the example in Figure 5, there are four non-zero probability basis states "0000", "0010", "1101" and "1110", condensing the $0th$ (0000), $2nd$ (0010), $13th$ (1101) and $14th$ (1110) rows of the matrix F. ❷ Then, we can identify the operands of the tensor product based on the row indices and the grouping scheme of qubits. For example, the row index of $F(2,3)$ is $2 = |0010\rangle$. According to the (2,1,1) grouping scheme, we can trace the indices of the operand in the matrix A.

$$|00\rangle \rightarrow A_1(0,3),\ |1\rangle \rightarrow A_2(1,3),\ |0\rangle \rightarrow A_3(0,3) \tag{7}$$

❸ By compressing the matrix F into a row-indexed dense matrix, it still enables the parallelization in the tensor-product and computing the non-zero outputs. ❹ Finally, we perform the MAC operator between the columns of the matrix F and the non-zero probability basis states in $\vec{p}_{noisy}$, following Equation (6). When tracing the non-zero elements, the tile-based parallelization strategy allows us to reuse indices and operands, alleviating the frequent updating of output probability. Specifically, for the values at the same row of a tile, the required row indices for their operands are identical. For instance, both $F(2,2)$ and $F(2,3)$ share the indices $|0010\rangle$ in Figure 5. Consequently, each tile can reuse a decoded set of indices for T_m times. Similarly, for values within the same column of a tile, the required operands originate from the same index. For example, the tensor-product for both $F(0,2)$ and $F(2,2)$ use $A_1(0,2)$ and $A_3(0,2)$. Therefore, each tile can reuse the results from matrix-vector multiplication for T_n values.

4.3 Leveraging Input Sparsity

Due to bit-level sparsity (opportunity II), we can transform the MVM operator to select a single column from the matrix. However, the input for mitigating computations is the basis state. To leverage this sparsity, ❶ we need to obtain corresponding sub-basis states from the basis states based on grouping scheme. For example, according to the (2,1,1) grouping scheme, $|1101\rangle$ is divided into: $|1101\rangle \rightarrow |11\rangle, |0\rangle, |1\rangle$. ❷ Due to its normalization property, the number represented by the basis state points to the position where the value is 1 in the basis vector. For instance, the '11' in $|11\rangle$ represents the number 3, indicating that the $3rd$ element of $|11\rangle$ is 1,

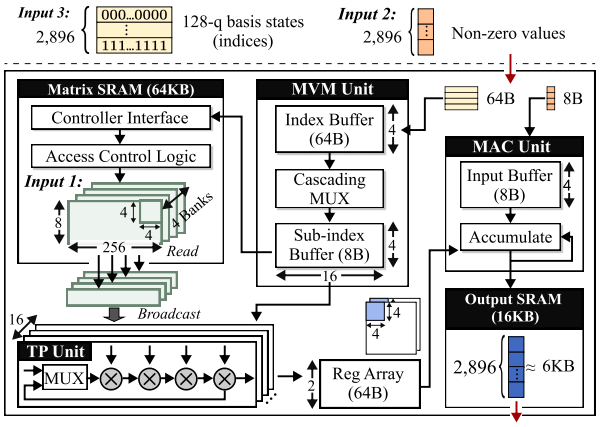


Figure 6: Overview of SiTA accelerator.

$|11\rangle = [0, 0, 0, 1]$. ❸ Based on the position of '1', we can directly extract columns of the matrix based on the basis state to complete the MVM operator.

$$\begin{aligned} M_1 |11\rangle &= M_1 (:, 3) = A_1 (:, 2) \\ M_2 |0\rangle &= M_2 (:, 0) = A_2 (:, 2) \\ M_3 |1\rangle &= M_3 (:, 1) = A_3 (:, 2) \end{aligned} \tag{8}$$

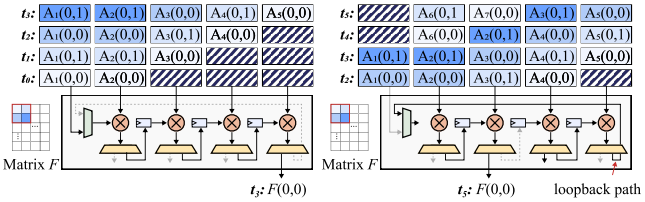
5 Accelerator Design

5.1 Overall Architecture

Because the sparse dataflow of tensor-product operations requires dynamically selecting the computation path based on the inputs, we design a specialized accelerator to achieve fast and accurate quantum readout error mitigation.

Memory Hierarchy. The inputs include 1) the mitigation matrices, 2) the non-zero noisy probability, and 3) the basis states of this non-zero probability. After partitioning the qubit into multiple groups, the mitigation matrices become relatively small and can be pre-stored in the on-chip memory. On the other hand, the non-zero noisy probability is dynamic and varies according to the circuit and measured qubits. Thus, we choose to stream the non-zero noisy probability and its basis states via two separate channels. The on-chip memory system includes two main SRAMs: one for the mitigation matrix (64KB) and another for the mitigated probability vector (16KB), which allows a maximum of 8K non-zero values, sufficient to accommodate results for the current quantum algorithms and quantum devices.

Figure 6 gives an example of the memory behavior for dealing with a 128-qubit BV algorithm, where the noisy probability vector contains 2,896 non-zero values. We adopt the grouping scheme that divides the 128 bits into 64 groups, resulting in 64 4×4 matrices for iterative tensor-product. The mitigation matrix necessitates 16KB (4×2×2KB) on-chip memory, which contains two sets of 64×4×4 mitigation matrices. In a single cycle, four basis states enter the 64B index buffer (128b×4) for the decoding stage. The decoding result serves as the read signal to fetch column vectors from the SRAM matrix. After completing the tensor-product calculation via the dynamic multiplier chain, 64B (2×16×16b) of memory is required for the tiles of two intermediate matrices. The final result is transmitted off-chip through a 16KB output SRAM.



(a) 8-qubit system (5 groups) (b) 16-qubit system (7 groups)

Figure 7: Details of the dynamic multiplier chain.

Computing Modules. The *MVM unit* consists of two register arrays and one cascading multiplexer to exploit the bit-level sparsity. It reads binary strings of a series of basis states and tells which column in the mitigation matrix needs to be sent (Equation (8)) to implement the column-selection operator. The *TP unit* is the key compute engine that involves multiple PEs, which receives the columns from the mitigation matrix and selects the elements according to the qubit grouping scheme (Equation (7)). Then, this unit parallelizes the sparse tensor-product dataflow in terms of probability-level and state-level parallelism. The *MAC unit* receives the results from the TP unit in a tile-by-tile manner. Each tile is multiplied by the noise probability vector. Finally, the results from different tiles are gathered to output the mitigated vector.

5.2 Dynamic Multiplier Chain in PE Design

The number of qubits and grouping scheme vary across quantum devices, leading to different multiplication counts in the TP operator. To adapt to different quantum devices, we design a dynamic multiplier chain consisting of four multipliers for computation, one multiplexer, and four demultiplexers for reconfiguration. Their configuration signals are generated by software-level controls based on the number of qubits and the corresponding grouping strategy, ensuring the generation of appropriate control signals. Specifically, the demultiplexer determines whether the current computation result is directly output or transferred to the next multiplier through registers, while the multiplexer decides whether to feed back the intermediate result from the previous stage to the multiplier input, thereby forming a loop structure to support successive multiplications. The overall data flow propagates among multiple multipliers in a systolic array manner, enabling efficient pipelined tensor-product computation. In Figure 7 (a), four multiplications are required, so no feedback loop is needed; the computation of one tile of the intermediate matrix F can be completed through pipelined data transfer. In contrast, for the 16-qubit case in Figure 7 (b), six multiplications are required, which are performed through a feedback loop, demonstrating the flexibility of the multiplier chain.

6 Evaluation

6.1 Experimental Setup

Hardware Implementation and Baselines. SiTA is implemented in Xilinx HLS and evaluated on a Xilinx Alveo U50 platform at 300 MHz. We compare SiTA with the SOTA readout error mitigation methods, including IBU [23], Mthree [17], QuFEM [25], and SpREM [32]. QuFEM works on AMD EPYC 9554 CPU, while IBU and Mthree works on A100 GPU. SpREM is a hardware-level technique implemented on the Alveo U50 platform at 300 MHz.

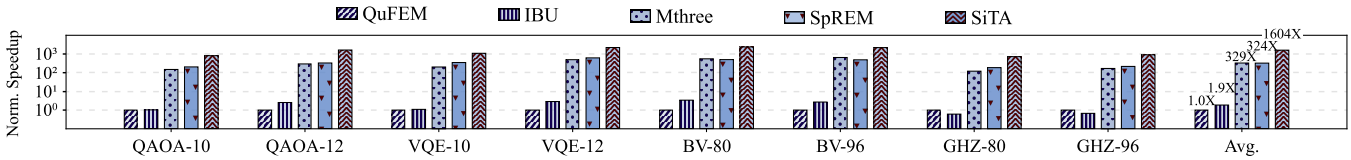


Figure 8: Performance comparison with SOTA error mitigation methods.

Table 1: Hellinger fidelity comparison on the Rigetti device.

Workload	Raw	Mthree [17]	SpREM [32]	IBU [23]	QuFEM [25]	SiTA
BV-16	0.22	0.47	0.48	0.60	0.61	**0.68**
GHZ-16	0.20	0.34	0.35	0.35	0.45	**0.49**
QAOA-10-2L	0.19	0.20	0.20	0.21	0.25	**0.29**
QAOA-12-2L	0.12	0.13	0.13	0.15	0.19	**0.24**
VQE-10-2L	0.39	0.40	0.41	0.42	0.48	**0.51**
VQE-12-2L	0.28	0.28	0.29	0.31	0.34	**0.35**
FAL.-10-35L	0.37	0.38	0.38	0.39	0.40	**0.42**
FAL.-12-30L	0.23	0.25	0.26	0.28	0.31	**0.32**

Mitigation Matrix and Benchmarks. We characterize the mitigation matrices using QuFEM [25] on the 79-qubit Rigetti quantum platform [22], which shows average readout, CZ gate, and XY gate errors of 5.5%, 5.2%, and 4.4%, respectively. The benchmarks include BV, GHZ, FALQON [14], VQE [9], and QAOA [7].

6.2 Evaluation of Mitigation Latency

Figure 8 presents the speedup of SiTA over the baselines. SiTA achieves an average speedup of 5.0×, 4.9×, 844×, and 1604× over SpREM [32], Mthree [17], IBU [23], and QuFEM [25], respectively. These speedups are achieved by leveraging sparsity to eliminate redundant computations. Specifically, SiTA features probability-level and state-level parallelism to accelerate the tensor-product operator and leverages sparsity to eliminate the computation involving zero values. Consequently, for algorithms with significant sparsity (e.g., the 80-qubit BV algorithm), SiTA achieves up to a 2381× speedup over QuFEM.

6.3 Evaluation of Mitigation Fidelity

For clarity, we simplify the representation of the variational quantum algorithm. For instance, *QAOA-10-2L* refers to a 10-qubit QAOA algorithm with two layers of quantum gates. In Table 1, SiTA achieves an average fidelity improvement of 1.1× to 1.4× compared to the baselines. For the three variational quantum algorithms, their quantum circuits contain more two-qubit gates, leading to increased crosstalk noise during the readout process. However, both IBU [23] and Mthree [17] fail to account for crosstalk, while SpREM [32] only models local crosstalk. As a result, the error mitigation effects of Mthree, IBU, and SpREM are low and even ineffective (e.g., the 12-qubit VQE algorithm). In contrast, SiTA mitigates the crosstalk noise between different qubits via the iterative tensor-product formulation, achieving an average fidelity improvement of 43.5% compared to the original fidelity.

6.4 Accelerator Performance over GPU

To further compare performance with GPUs, we apply the pruning technique from SpREM [32] to the mitigation matrix, store it in CSR sparse format, and accelerate it using the cuSPARSE [18] on A100 and H100 GPUs. In Figure 9, SiTA delivers 11.2× and 5.3× speedups over the A100 and H100 GPUs, respectively. Moreover, as the scale

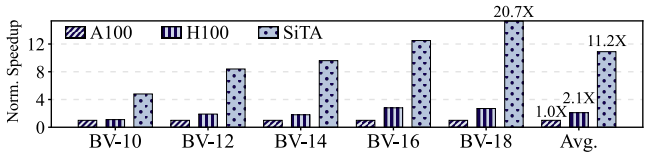


Figure 9: Error mitigation speedup over GPU.

of the quantum algorithm increases, the speedup also improves. For example, the 18-qubit BV algorithm achieves speedups of 20.7× and 7.7× over the A100 and H100, respectively. This improvement is attributed to SiTA 's exploitation of output sparsity, which enables dynamic selective computation and sparsification tailored to specific algorithms. In contrast, GPUs are more general-purpose, capable of mitigating errors for any algorithm, but this universality comes at the cost of reduced acceleration, highlighting the necessity of dedicated hardware.

7 Related Work

Sparsity in Tensor Computation. In tensor computations, most sparse structures are irregular. Some sparse accelerators [8, 31] improve the performance of general matrix multiplication by employing different sparse dataflows, such as inner product in SIGMA [20] and outer product in HARP [10]. Furthermore, structural sparsity exists in tensor computations for specific applications. For example, some works [12, 21, 29] prunes the weak connections between tokens to accelerate the inference.

Quantum Readout Error Mitigation. In terms of software, some works mitigate readout crosstalk errors by measuring subsets of qubits [5, 6, 11, 26]. Hammer [27] exploits the pattern of errors by modeling it as a Hamming-weighted graph, while Q-BEEP [24] further enhances this modeling using an iterative Bayesian network. At the hardware level, HERQULES [15] designs a discriminator that improves single-shot qubit-state discrimination using readout frequency trajectories. SpREM [32] compresses the mitigation matrix by exploiting Hamming sparsity.

8 Conclusion

Quantum computing relies on classical computers to perform auxiliary operations that mitigate quantum readout noise. However, existing mitigation approaches overlook the mitigation speed, limiting the potential for end-to-end acceleration. In this paper, we propose SiTA to enhance mitigation speed through the co-design of software and hardware without sacrificing accuracy. Specifically, we leverage the inherent sparsity of Hilbert space to reduce redundant computation. In addition, we feature probability-level and state-level parallelism to accelerate the mitigation process. Experiments demonstrate that SiTA achieves orders of magnitude speed improvement over the conventional hardware.

References

[1] A Burrell. 2010. High fidelity readout of trapped ion qubits. Ph. D. Dissertation. Oxford University, UK.

[2] Yanzhu Chen, Maziar Farahzad, Shinjae Yoo, and Tzu-Chieh Wei. 2019. Detector Tomography on IBM Quantum Computers and Mitigation of an Imperfect Measurement. Phys. Rev. A 100 (Nov 2019), 052315. Issue 5. doi:10.1103/PhysRevA.100.052315

[3] J. M. Chow, L. DiCarlo, J. M. Gambetta, A. Nunnenkamp, Lev S. Bishop, L. Frunzio, M. H. Devoret, S. M. Girvin, and R. J. Schoelkopf. 2010. Detecting Highly Entangled States with a Joint Qubit Readout. Phys. Rev. A 81 (Jun 2010), 062325. Issue 6. doi:10.1103/PhysRevA.81.062325

[4] Jerry M. Chow, Jay M. Gambetta, A. D. Córcoles, Seth T. Merkel, John A. Smolin, Chad Rigetti, S. Poletto, George A. Keefe, Mary B. Rothwell, J. R. Rozen, Mark B. Ketchen, and M. Steffen. 2012. Universal Quantum Gate Set Approaching Fault-Tolerant Thresholds with Superconducting Qubits. Phys. Rev. Lett. 109 (Aug 2012), 060501. Issue 6. doi:10.1103/PhysRevLett.109.060501

[5] Siddharth Dangwal, Gokul Subramanian Ravi, Poulami Das, Kaitlin N. Smith, Jonathan Mark Baker, and Frederic T. Chong. 2024. VarSaw: Application-tailored Measurement Error Mitigation for Variational Quantum Algorithms. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4 (ASPLOS) (ASPLOS '23). Association for Computing Machinery, New York, NY, USA, 362–377. doi:10.1145/3623278.3624764

[6] Poulami Das, Swamit Tannu, and Moinuddin Qureshi. 2021. JigSaw: Boosting Fidelity of NISQ Programs via Measurement Subsetting. In MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO) (MICRO '21). Association for Computing Machinery, New York, NY, USA, 937–949. doi:10.1145/3466752.3480044

[7] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. arXiv preprint arXiv:1411.4028 (2014).

[8] Gerasimos Gerogiannis, Serif Yesil, Damitha Lenadora, Dingyuan Cao, Charith Mendis, and Josep Torrellas. 2023. Spade: A Flexible and Scalable Accelerator For SpMM and SDDMM. In Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA). 1–15.

[9] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. 2017. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. nature 549, 7671 (2017), 242–246.

[10] Jinkwon Kim, Myeongjae Jang, Haejin Nam, and Soontae Kim. 2023. HARP: Hardware-Based Pseudo-Tiling for Sparse Matrix Multiplication Accelerator. In Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 1148–1162.

[11] Peiyi Li, Ji Liu, Alvin Gonzales, Zain Hamid Saleem, Huiyang Zhou, and Paul Hovland. 2024. QuTracer: Mitigating Quantum Gate and Measurement Errors by Tracing Subsets of Qubits. In 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA). 103–117. doi:10.1109/ISCA59077.2024.00018

[12] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. 2021. Sanger: A Co-design Framework for Enabling Sparse Attention using Reconfigurable Architecture. In MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 977–991.

[13] Filip B Maciejewski, Zoltán Zimborás, and Michał Oszmaniec. 2020. Mitigation of readout noise in near-term quantum devices by classical post-processing based on detector tomography. Quantum 4 (2020), 257.

[14] Alicia B Magann, Kenneth M Rudinger, Matthew D Grace, and Mohan Sarovar. 2022. Feedback-based quantum optimization. Physical Review Letters 129, 25 (2022), 250502.

[15] Satvik Maurya, Chaithanya Naik Mude, William D. Oliver, Benjamin Lienhard, and Swamit Tannu. 2023. Scaling Qubit Readout with Hardware Efficient Machine Learning Architectures. In Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 7, 13 pages. doi:10.1145/3579371.3589042

[16] Seth T. Merkel, Jay M. Gambetta, John A. Smolin, Stefano Poletto, Antonio D. Córcoles, Blake R. Johnson, Colm A. Ryan, and Matthias Steffen. 2013. Self-consistent Quantum Process Tomography. Phys. Rev. A 87 (Jun 2013), 062119. Issue 6. doi:10.1103/PhysRevA.87.062119

[17] Paul D. Nation, Hwajung Kang, Neereja Sundaresan, and Jay M. Gambetta. 2021. Scalable Mitigation of Measurement Errors on Quantum Computers. PRX Quantum 2 (Nov 2021), 040326. Issue 4. doi:10.1103/PRXQuantum.2.040326

[18] NVIDIA. [n. d.]. GPU library APIs for sparse computation. https://developer.nvidia.com/cusparse

[19] NVIDIA. 2025. Basic Linear Algebra on NVIDIA GPUs. https://developer.nvidia.com/cublas

[20] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. Sigma: A sparse and Irregular Gemm Accelerator with Flexible Interconnects for DNN Training. In 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 58–70.

[21] Yubin Qin, Yang Wang, Dazheng Deng, Zhiren Zhao, Xiaolong Yang, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. 2023. Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction. In Proceedings of the 50th Annual International Symposium on Computer Architecture. 1–14.

[22] Rigetti. 2025. Rigetti Computing: Quantum Computing. https://www.rigetti.com/

[23] KJ Satzinger, Y-J Liu, A Smith, C Knapp, M Newman, C Jones, Z Chen, C Quintana, X Mi, A Dunsworth, et al. 2021. Realizing Topologically Ordered States on a Quantum Processor. Science 374, 6572 (2021), 1237–1241. doi:10.1126/science.abi8378

[24] Samuel Stein, Nathan Wiebe, Yufei Ding, James Ang, and Ang Li. 2023. Q-BEEP: Quantum Bayesian Error Mitigation Employing Poisson Modeling over the Hamming Spectrum. In Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 8, 13 pages. doi:10.1145/3579371.3589043

[25] Siwei Tan, Liqiang Lu, Hanyu Zhang, Jia Yu, Congliang Lang, Yongheng Shang, Xinkui Zhao, Mingshuai Chen, Yun Liang, and Jianwei Yin. 2024. QuFEM: Fast and Accurate Quantum Readout Calibration Using the Finite Element Method. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 948–963. doi:10.1145/3620665.3640380

[26] Wei Tang, Teague Tomesh, Martin Suchara, Jeffrey Larson, and Margaret Martonosi. 2021. CutQC: Using Small Quantum Computers for Large Quantum Circuit Evaluations. In Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 473–486. doi:10.1145/3445814.3446758

[27] Swamit Tannu, Poulami Das, Ramin Ayanzadeh, and Moinuddin Qureshi. 2022. HAMMER: Boosting Fidelity of Noisy Quantum Circuits by Exploiting Hamming Behavior of Erroneous Outcomes. In Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (ASPLOS '22). Association for Computing Machinery, New York, NY, USA, 529–540. doi:10.1145/3503222.3507703

[28] Chenning Tao, Liqiang Lu, Size Zheng, Li-Wen Chang, Minghua Shen, Hanyu Zhang, Fangxin Liu, Kaiwen Zhou, and Jianwei Yin. 2025. Qtenon: Towards Low-Latency Architecture Integration for Accelerating Hybrid Quantum-Classical Computing. In Proceedings of the 52nd Annual International Symposium on Computer Architecture. 299–312.

[29] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. Spatten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. In 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE, 97–110.

[30] Yilun Xu, Gang Huang, Jan Balewski, Ravi Naik, Alexis Morvan, Bradley Mitchell, Kasra Nowrouzi, David I Santiago, and Irfan Siddiqi. 2021. QubiC: An open-source FPGA-based control and measurement system for superconducting quantum information processors. IEEE Transactions on Quantum Engineering 2 (2021), 1–11.

[31] Yifan Yang, Joel S Emer, and Daniel Sanchez. 2024. Trapezoid: A versatile accelerator for dense and sparse matrix multiplications. In 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA). IEEE, 931–945.

[32] Hanyu Zhang, Liqiang Lu, Siwei Tan, Size Zheng, Jia Yu, and Jianwei Yin. 2024. SpREM: Exploiting Hamming Sparsity for Fast Quantum Readout Error Mitigation. In Proceedings of the 61st ACM/IEEE Design Automation Conference. 1–6.

[33] Han-Sen Zhong, Hui Wang, Yu-Hao Deng, Ming-Cheng Chen, Li-Chao Peng, Yi-Han Luo, Jian Qin, Dian Wu, Xing Ding, Yi Hu, Hu Peng, Xiao-Yan Yang, Wei-Jun ZHANG, Hao Li, Yu-Xuan Li, Xiao Jiang, Lin Gan, Guangwen Yang, Li-Xing You, Zhen Wang, Li Li, Nai-Le Liu, Chao-Yang Lu, and Jian-Wei Pan. 2020. Quantum computational advantage using photons. Science 370, 6523 (2020), 1460–1463.